Documento comparativo — Entrega II (Genki Health)

Bases de Datos II — Universidad Rafael Landívar
Integrantes: Carlos Hugo Escobar Gómez · Héctor Jordin Adolfo García Coyoy
Evidencia: ADR-0001…0005 (`docs/specs/adr/`) · `DECISIONS.md` · `docs/ARCHITECTURE.md` · contratos C4/C5 (`docs/plan/contracts/`) · `SCHEMA.md` y `SECURITY-FREE.md` (`docs/plan/full-system/contracts/`) · DDL y adaptadores en `server/src/adapters/` · auditoría en `tests/e2e-audit/`. Las rutas de código son relativas a `server/src/adapters/` salvo que se indique otra raíz.

---

1. Asignación institución → motor

Fijada en ADR-0002 y `shared/src/engines.ts` (`INSTITUTION_ENGINE_ASSIGNMENTS`). Cada motor se usa una sola vez.

Institución	Motor	Volumen	Estructura	Transacciones estrictas
Hospital general	**PostgreSQL**	Alto (citas, diagnósticos, tratamientos, hospitalización)	Relacional normalizado (C4a: `consulta` + tablas hijas, admisiones, procedimientos)	Máxima: el paquete consulta+diagnóstico+tratamiento no puede quedar a medias (Entrega I §1.1)
Centro médico pediátrico	**MySQL**	Consultas frecuentes y cortas; seguimiento longitudinal (años)	Datos estructurados y repetitivos (crecimiento, vacunas, hitos); esquema C4b InnoDB	Alta en cierre atómico de consulta y escrituras de seguimiento; segundo motor relacional distinto al hospital
Laboratorio clínico	**MongoDB**	Órdenes/resultados acumulables (cientos de exámenes por paciente)	Estructura **variable** (`RESULTADO.datos`: hematología ≠ orina ≠ cultivo; Entrega I §5.4)	Sesiones multi-documento en réplica `rs0`; resultados como documentos referenciados, no array ilimitado bajo el paciente
Clínica odontológica	**Firestore**	Bajo/medio; flujos cortos de secretaria	Documento natural para odontograma/superficies anidadas (Entrega I §5.5)	Transacciones Admin SDK; historial referenciado, no embebido sin límite

Criterio (Entrega I §1.2): relacionales para información estable; documentales para esquema variable (lab) o anidación operativa (odontología).

Por qué cada motor encaja con su institución:

- **Hospital → PostgreSQL.** Es la institución con más volumen y con la regla más dura (“no puede perderse un diagnóstico a medias”). PostgreSQL da transacciones ACID con `RETURNING`, bloqueo de fila `SELECT … FOR UPDATE OF`, tipos `TIMESTAMPTZ/NUMERIC` e índices parciales. Además concentra la identidad y las sesiones de toda la plataforma (ADR-0005).
- **Pediatría → MySQL.** Los datos son tabulares y repetitivos (mediciones, dosis, hitos) y no cambian de forma. InnoDB ofrece ACID y llaves foráneas en un segundo motor relacional distinto del hospital, y permite comparar dialectos SQL con las mismas reglas de negocio (§6).
- **Laboratorio → MongoDB.** El resultado de cada examen tiene campos distintos según su tipo. Un documento con validador `$jsonSchema` (`mongodb/schema.ts`, `mongodb/schema.json`) admite esa variación sin crear una tabla por tipo de examen. Las transacciones multi-documento de la réplica `rs0` siguen garantizando el cierre atómico.
- **Odontología → Firestore.** La clínica es pequeña y no tiene personal técnico (Entrega I §1.1). Un servicio documental administrado no requiere operar un servidor de base de datos, y el odontograma tiene forma de documento anidado. El acceso pasa siempre por el backend con Admin SDK (ADR-0004).

---

2. Modelado por motor

PostgreSQL (Hospital)

SQL a mano, sin ORM (ADR-0001, driver `pg`). El núcleo es paciente/expediente/consulta más las tablas hijas `diagnostico`, `tratamiento`, `receta` y `detalle_receta`, normalizadas en 3FN desde Entrega I. Las llaves son `GENERATED BY DEFAULT AS IDENTITY` y las fechas, `TIMESTAMPTZ` (`postgres/migrations/0001-core.sql`). La unicidad del documento es `uq_paciente_documento(id_institucion, numero_documento)`, y "un solo responsable principal" es el índice parcial `uq_responsable_principal … WHERE es_principal`. Identidad y refresh sessions viven **solo** en este motor (ADR-0005). La hospitalización tiene tablas propias.

MySQL (Pediatría)

Mismo núcleo clínico en InnoDB (`mysql/migrations/`), con `AUTO_INCREMENT`, `DATETIME(3)` y `utf8mb4_0900_as_ci`. El mapper convierte DATETIME/DECIMAL. MySQL no tiene índices parciales, así que el responsable principal único se modela con una columna generada `principal_paciente … STORED` más `UNIQUE KEY` (`mysql/migrations/0001-core.sql`). Los responsables son filas separadas que se crean en la misma transacción del alta. Las extensiones pediátricas (crecimiento, vacunas, hitos) viven aquí. No guarda hashes de contraseña.

MongoDB (Laboratorio) — embed vs referencia (ADR-0003, C5.1)

	Decisión	Justificación
Embed	En documento `consultas`: `diagnosticos[]`, `tratamientos[]`, `receta`+`detalles[]` (≤256 KiB UTF-8)	Un paquete clínico atómico y acotado (1–10 dx, 1–20 tx, 0–20 detalles)
Referencia	Consultas **no** se embeben en paciente/expediente	Historial unbounded
Referencia	Catálogos por id entero; `responsables` colección aparte (alta multi-doc con rollback)	Sin FK nativas; prueba de transacción
Referencia	`ordenes_examen` + `resultados` (`full/lab-results.ts`); `datos` variable ligado a la orden	Un paciente acumula cientos de exámenes → no embeber bajo paciente

`_id` = `<institution>:<numeric id>`. Los validadores `$jsonSchema` y los índices únicos compuestos por institución (`uq_inst_numero_documento`, y el índice parcial `partialFilterExpression: { es_principal: true }` para el responsable principal) están en `mongodb/schema.ts`.

Firestore (Odontología) — embed vs referencia (ADR-0004, C5.2)

	Decisión	Justificación
Embed	En `consultations/{id}`: mismos arrays clínicos (≤256 KiB; margen bajo 1 MiB)	Mismo bundle atómico
Referencia	Colecciones hijas bajo `institutions/{id}`: patients, records, consultations…	Historial unbounded
Referencia	Unicidad de documento vía `documentKeys/{sha256}` en la tx de alta	El número puede contener `/`
Embed acotado	Líneas de plan / superficies dentales con techos (SCHEMA.md)	Coincide con el odontograma en papel; sesiones se referencian

React no usa el SDK de Firestore; solo Admin en el backend.

3. Consistencia (registrar consulta + diagnóstico + receta)

Caso de uso compartido: `closeConsulta` (`application/close-consulta.ts`). Misma regla en los cuatro motores: **todo el bundle o nada**.

Motor	Mecanismo	Evidencia en código
PostgreSQL	`BEGIN ISOLATION LEVEL READ COMMITTED` → `COMMIT` / `ROLLBACK`	`postgres/unit-of-work.ts`; `PostgresConsultaRepository` inserta `consulta` → `diagnostico` → `tratamiento` → `receta`/`detalle_receta` dentro de `runInTransaction`
MySQL	Isolation READ COMMITTED + `BEGIN`/`COMMIT`/`ROLLBACK`	`mysql/unit-of-work.ts`; mismo grafo INSERT en `MysqlConsultaRepository` con bloqueo de expediente
MongoDB	`startSession` + `startTransaction` (`readConcern: snapshot`, `writeConcern: majority`), ≤3 reintentos ante errores transitorios	`mongodb/unit-of-work.ts`; un `insertOne` de consulta **con** dx/tx/receta embebidos tras validar referencias en sesión
Firestore	`db.runTransaction` (Admin); lecturas antes de escrituras; ≤3 intentos	`firebase/unit-of-work.ts` + `consulta-repository.ts` (`set` del documento `consultations/{id}` con bundle)

La suite de contrato C3.4 exige rollback nativo: por ejemplo, un medicamento inválido no deja una consulta parcial.

Las carreras con “archivar expediente” se resuelven distinto en cada motor. En SQL, `SELECT … FOR UPDATE OF e` bloquea la fila del expediente. En MongoDB, `findOneAndUpdate` sobre `_rev` hace de control optimista. En Firestore, `records.revision` se incrementa dentro de la transacción y Firestore reintenta si hubo conflicto.

---

4. Seguridad por motor

Clasificación (Entrega I §9 / ADR-0005 / C5.3): **pública** (catálogos, nombres de institución), **confidencial** (identidad/contacto paciente, estado de expediente), **restringida** (contenido clínico; hashes/JTI). Campo no clasificado → restringido. Roles API MVP: `administrador`, `recepcion`, `medico`, `auditor`. Auth: email+password, Argon2id, JWT access+refresh; SoR en Hospital PG.

Motor	Aplicación de seguridad
PostgreSQL	Roles NOLOGIN `genki_app` / `genki_identity` / `genki_report` (`postgres/security.sql`). `REVOKE ALL FROM PUBLIC`. App: SELECT catálogos; SELECT/INSERT clínico; UPDATE acotado a expediente/contadores. Identity toca `usuario`/`auth_session`, no clínico. Report: solo vista agregada. DSN runtime ≠ dueño de migración.
MySQL	Roles `genki_app` / `genki_report` (`mysql/security.sql`); vista DEFINER; runtime ≠ root; alcance por institución en repositorio; denegaciones nativas en `test/security/mysql.test.ts`.
MongoDB	Roles `genkiApp` / `genkiReporter` (`mongodb/security.ts`): find/insert clínico, update expediente/counters; reporter solo vista; sin remove/drop/bypassDocumentValidation; auth + keyfile rs0.
Firestore	Reglas deny-by-default (`infra/firebase/firestore.rules`); solo reporter lee `reportes`. Admin SDK bypasea reglas → seguridad efectiva = JWT HTTP + puertos scoped a institución 4 (no se afirma que las reglas protejan Admin).

Las contraseñas de los seeds se leen solo de `SEED_PASSWORD` (variable de entorno). Los puertos de los contenedores se publican solo en `127.0.0.1` (`infra/local-isolated/docker-compose.isolated.yml`). No se afirma TLS ni cifrado en reposo para los motores locales ni para el emulador (ADR-0005).

4.1 Cifrado en reposo: fuera del alcance del MVP

Entrega I §9.4 previó pgcrypto para columnas restringidas en PostgreSQL, tablespace encryption en MySQL y field-level encryption en MongoDB. ADR-0005, `DOCUMENTS-SECURITY.md` (C5) y el supuesto A5 de `AGENTS.md`

dejan esa aspiración **diferida**: no se implementó y este documento no afirma que exista. `SECURITY-FREE.md` describe un diseño futuro sin costo, pero es plan y no código. La tabla indica qué usaría cada motor:

Motor (imagen)	Mecanismo que se usaría	Estado en el MVP
PostgreSQL (`postgres:16-alpine`)	PostgreSQL comunitario no trae TDE nativo: cifrado del disco o volumen del host, y `pgcrypto` (`pgp_sym_encrypt`) para columnas restringidas puntuales	No activado
MySQL (`mysql:8.4`)	Cifrado de datos en reposo de InnoDB (`ENCRYPTION='Y'` por tablespace) con un componente de keyring (p. ej. `component_keyring_file`) que custodia la llave maestra	No activado
MongoDB (`mongo:7`, Community)	El Encrypted Storage Engine es exclusivo de Enterprise. En Community quedan el cifrado del volumen y el cifrado de campo explícito con CSFLE o Queryable Encryption (libmongocrypt, con bóveda de llaves propia)	No activado
Firestore	En modo real (`FIRESTORE_MODE=real`, valor por defecto de `infra/local-isolated/start.sh`), Google Cloud cifra los datos en reposo por defecto con llaves administradas por Google. CMEK (llaves del cliente) sería opcional y no está configurado. El emulador local no ofrece esa garantía	Por defecto en el servicio real

Se difiere por cuatro razones. Los datos son sintéticos (A13). Los motores corren en volúmenes Docker locales (`genki_mvp_pgdata`, `genki_mvp_mysqldata`, `genki_mvp_mongodata`). El proyecto no usa servicios pagos (Enterprise, KMS administrado). Y la gestión de llaves (keyring, key vault, rotación y respaldo de la llave maestra) es un problema operativo propio que el MVP no cubre. Mientras tanto, el riesgo se mitiga con los roles de mínimo privilegio, la clasificación de campos, Argon2id y los puertos ligados a loopback descritos arriba.

---

5. Hexágono y enrutamiento

Arquitectura hexagonal (ADR-0001, `ARCHITECTURE.md`): `application/` y `domain/` dependen de **puertos**; cada motor es un adaptador (`postgres` | `mysql` | `mongodb` | `firebase`). Sin ORM.

Al autenticarse, el JWT lleva `institucion_id`, `institucion_kind` y `motor`; el registro (`compose-engines` / adapter registry) elige el `AdapterSet`. Las mismas rutas REST y la misma UI React/Vite sirven las cuatro instituciones: cambian los adaptadores, no las reglas de negocio ni la interfaz. El superadministrador de evaluación puede seleccionar institución y recibir un JWT reemitido.

---

6. Consultas equivalentes en los cuatro motores

Cada operación implementa el mismo método de puerto (`PacienteRepositoryPort`, `ConsultaRepositoryPort`) en los cuatro adaptadores. Los extractos son reales pero abreviados: se recortaron listas de columnas y el filtro opcional de estado (…).

6.1 Buscar paciente por documento o nombre (`*/paciente-repository.ts`, método `list`)

PostgreSQL

```
SELECT p.*, e.id_expediente, e.numero_expediente, e.estado AS expediente_estado
  FROM paciente p
  JOIN expediente e ON e.id_paciente = p.id_paciente
 WHERE p.id_institucion = $1
   AND ( p.numero_documento = $2 OR e.numero_expediente = $3
         OR concat_ws(' ', p.primer_nombre, NULLIF(p.segundo_nombre, ''),
                      p.primer_apellido, NULLIF(p.segundo_apellido, ''))
            ILIKE $4 ESCAPE '\' )
 ORDER BY p.id_paciente ASC
 LIMIT $5 OFFSET $6
```

MySQL

```
SELECT p.*, e.id_expediente, e.numero_expediente, e.estado AS expediente_estado
  FROM paciente p
  INNER JOIN expediente e ON e.id_paciente = p.id_paciente
 WHERE p.id_institucion = ?
   AND (p.numero_documento = ? OR e.numero_expediente = ?
        OR LOWER(TRIM(CONCAT_WS(' ', NULLIF(TRIM(p.primer_nombre), ''), …)))
           LIKE LOWER(?) ESCAPE '\\')
 ORDER BY p.id_paciente ASC
 LIMIT ? OFFSET ?
```

Los dos dialectos difieren en tres puntos. PostgreSQL usa parámetros posicionales `$n` y MySQL usa `?`. La búsqueda sin mayúsculas se hace con `ILIKE` en PostgreSQL y con `LOWER(…) LIKE LOWER(?)` en MySQL (la intercalación `utf8mb4_0900_as_ci` ya ignora mayúsculas). Y MySQL trata `\` como escape dentro del literal, por eso el `ESCAPE` se escribe doble. La búsqueda exacta (`findByDocumento`) es `WHERE id_institucion = … AND numero_documento = …` en ambos y usa el índice único `uq_paciente_documento`.

MongoDB (`mongodb/paciente-repository.ts`)

```
// findByDocumento: igualdad sobre el índice único uq_inst_numero_documento
db.collection('pacientes').findOne(
  { institution_id: toBsonInt(id_institucion), numero_documento: normalizeDocumento(n) },
  sessionOpts());
// list: lee pacientes y expedientes de la institución; filtra y pagina en el adaptador
const [pacientes, expedientes] = await Promise.all([
  db.collection('pacientes').find({ institution_id }, opts).toArray(),
  db.collection('expedientes').find({ institution_id }, opts).toArray(),
]);
```

El `WHERE id_institucion` se vuelve el campo `institution_id` en cada filtro, y el JOIN se vuelve un mapa en memoria porque expediente es una colección referenciada. La coincidencia por nombre (`matchesQuery`, subcadena sin mayúsculas) y la paginación se hacen en el adaptador, sin `$regex` ni agregación. Esto basta para el volumen del MVP, pero crece con el tamaño de la institución.

Firestore (`firebase/paciente-repository.ts`, `firebase/unit-of-work.ts`)

```
// unicidad: documentKeys/{sha256(numero)} se lee y se crea en la misma transacción
if ((await uow.read('documentKeys', key)) || (await this.findByDocumento(tipo, numero)))
  throw new ClinicalError('duplicate_document', 'Documento ya registrado en esta institución');
// búsqueda: lectura transaccional de la subcolección y filtro en el backend
const rows = await uow.query('patients', (row) => row.numero_documento === numero);
//   → transaction.get(db.collection(`institutions/${id}/patients`))
```

El alcance por institución lo da la **ruta** (`institutions/{id}/…`), no un campo. Firestore no tiene restricciones `UNIQUE` ni búsqueda por subcadena, así que la unicidad usa un documento-llave y el filtro por nombre corre en el backend.

6.2 Cerrar consulta con diagnóstico, tratamiento y receta (`*/consulta-repository.ts` + `*/unit-of-work.ts`)

PostgreSQL

```
BEGIN ISOLATION LEVEL READ COMMITTED;                    -- unit-of-work.ts
SELECT e.estado FROM expediente e JOIN paciente p ON p.id_paciente = e.id_paciente
 WHERE e.id_expediente = $1 AND p.id_institucion = $2 FOR UPDATE OF e;
INSERT INTO consulta (id_expediente, id_profesional, …, estado)
VALUES ($1, $2, …, $11) RETURNING *;
INSERT INTO diagnostico (id_consulta, codigo_cie10, …) VALUES ($1, $2, …) RETURNING *;
INSERT INTO tratamiento (id_diagnostico, …) VALUES ($1, …) RETURNING *;
INSERT INTO receta (…) RETURNING *;  INSERT INTO detalle_receta (…) RETURNING *;
COMMIT;                                                  -- o ROLLBACK ante cualquier error
```

MySQL

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  -- luego beginTransaction()
SELECT e.id_expediente, e.estado FROM expediente e
 INNER JOIN paciente p ON p.id_paciente = e.id_paciente
 WHERE p.id_institucion = ? AND e.id_expediente = ? FOR UPDATE OF e;
INSERT INTO consulta (…) VALUES (?, ?, NULL, ?, ?, ?, ?, ?, ?, ?, 'cerrada');
-- id: ResultSetHeader.insertId (el LAST_INSERT_ID() de la conexión)
INSERT INTO diagnostico (id_consulta, …) VALUES (?, …);   -- idem tratamiento/receta/detalle
COMMIT;                                                    -- o ROLLBACK
```

PostgreSQL recupera cada fila insertada con `RETURNING *`. MySQL no tiene `RETURNING`: toma el id generado de `insertId` y relee la fila si la necesita. MySQL además fija `time_zone = '+00:00'` por sesión porque `DATETIME` no guarda zona, mientras que PostgreSQL usa `TIMESTAMPTZ`. Una FK violada (`23503` en PostgreSQL) se traduce a `reference_not_found` y aborta toda la transacción.

MongoDB

```
session.startTransaction({ readConcern: { level: 'snapshot' }, writeConcern: { w: 'majority' } });
const exp = await db.collection('expedientes').findOne({ institution_id, id_expediente }, { session });
await db.collection('expedientes').findOneAndUpdate(          // control optimista vs. archivar
  { _id: exp._id, estado: 'activo', _rev: exp._rev }, { $set: { _rev: currentRev + 1 } }, { session });
await db.collection('consultas').insertOne({
  _id: `${inst}:${id_consulta}`, institution_id, id_expediente, /* … */
  diagnosticos, tratamientos, receta, detalles }, { session });   // un solo documento embebido
await session.commitTransaction();          // abortTransaction() y ≤3 reintentos transitorios
```

Los cinco INSERT de SQL se vuelven **un** `insertOne`, porque el paquete va embebido (ADR-0003). Sin llaves foráneas, profesional, CIE-10 y medicamentos se validan con `findOne` dentro de la misma sesión. El bloqueo de fila se reemplaza por el `_rev` del expediente.

Firestore

```
db.runTransaction(async (transaction) => {           // unit-of-work.ts, { maxAttempts: 3 }
  const record = await uow.read('records', id_expediente);         // transaction.get(ref)
  if (record.estado === 'archivado') throw new ClinicalError('expediente_archived', …);
  // lecturas de professionals / cie10 / medications / counters, todas antes de escribir
  await uow.update('records', id_expediente, { revision: record.revision + 1 });
  uow.set('consultations', id_consulta, { ...consulta, diagnosticos, tratamientos, receta, detalles });
  // al final: transaction.update(...) / transaction.set(...) de todas las escrituras en bloque
}, { maxAttempts: 3 });
```

Firestore exige que todas las lecturas ocurran antes que cualquier escritura. Por eso la unidad de trabajo acumula las escrituras y las envía al final, y Firestore reintenta la función completa si detecta un conflicto (concurrencia optimista). Antes de escribir se valida el techo de 256 KiB del documento.

6.3 Historial del expediente (`listByExpediente`)

PostgreSQL

```
SELECT c.* FROM consulta c
  JOIN expediente e ON e.id_expediente = c.id_expediente
  JOIN paciente p ON p.id_paciente = e.id_paciente
 WHERE c.id_expediente = $1 AND p.id_institucion = $2
 ORDER BY c.fecha_hora_inicio DESC, c.id_consulta DESC;
SELECT d.* FROM diagnostico d WHERE d.id_consulta = ANY($1::int[]) ORDER BY d.id_diagnostico;
-- igual para tratamiento, receta y detalle_receta: 4 consultas en paralelo
```

MySQL

```
SELECT c.* FROM consulta c INNER JOIN expediente e ON … INNER JOIN paciente p ON …
 WHERE p.id_institucion = ? AND c.id_expediente = ?
 ORDER BY c.fecha_hora_inicio DESC, c.id_consulta DESC;
-- por cada consulta (assembleCompleta):
SELECT * FROM diagnostico WHERE id_consulta = ? ORDER BY id_diagnostico ASC;
SELECT * FROM tratamiento WHERE id_diagnostico IN (?, ?, …) ORDER BY id_tratamiento ASC;
```

PostgreSQL pasa todos los ids como un solo arreglo (`ANY($1::int[])`). MySQL no tiene tipo arreglo, así que genera una lista `IN (?, …)` y arma cada consulta por separado.

MongoDB

```
db.collection('consultas')
  .find({ institution_id: toBsonInt(inst), id_expediente: toBsonInt(id) }, sessionOpts())
  .sort({ fecha_hora_inicio: -1, id_consulta: -1 })
  .toArray();          // cada documento ya trae diagnosticos/tratamientos/receta/detalles
```

Firestore

```
db.collection(`institutions/${inst}/consultations`)
  .where('id_expediente', '==', id)
  .orderBy('fecha_hora_inicio', 'desc')
  .orderBy('id_consulta', 'desc');     // índice compuesto: infra/firebase/firestore.indexes.json
// se ejecuta con transaction.get(query) dentro de runInTransaction
```

En los motores documentales, una sola lectura devuelve el paquete completo (embed), sin JOIN. Firestore necesita declarar el índice compuesto para combinar `where` y `orderBy`. En ningún motor se pagina el historial en el MVP: se devuelve completo, en orden descendente.

6.4 Resumen de diferencias

Aspecto	PostgreSQL	MySQL	MongoDB	Firestore
Parámetros	`$1, $2…`	`?`	objeto filtro	`where(campo, op, valor)`
Id generado	`IDENTITY` + `RETURNING *`	`AUTO_INCREMENT` + `insertId`	contador `counters` en la sesión	contador `counters` en la transacción
Texto sin mayúsculas	`ILIKE … ESCAPE '\'`	`LOWER() LIKE LOWER()` / intercalación `_ci`	filtro en el adaptador	filtro en el backend
Alcance institución	`p.id_institucion = $n` (JOIN)	`p.id_institucion = ?`	campo `institution_id` + `_id` prefijado	ruta `institutions/{id}/…`
Unicidad documento	`UNIQUE(id_institucion, numero_documento)`	ídem	índice único compuesto	doc `documentKeys/{sha256}`
Concurrencia vs. archivo	`FOR UPDATE OF e`	`FOR UPDATE OF e`	`_rev` optimista	`revision` + reintento de tx
Paquete clínico	5 tablas, 5+ INSERT	5 tablas, 5+ INSERT	1 documento embebido	1 documento embebido

---

7. Resultados de la auditoría automatizada

La auditoría vive en `tests/e2e-audit/` (ver su `README.md`). Corre contra el runtime real (`GENKI_MODE=engines`) en las cuatro instituciones, con datos de prueba prefijados `E2E-AUDIT-<run>`. Tiene tres partes:

- **Flujo completo** (`api-audit.mjs`): login y ruteo por motor, registro y búsqueda de paciente, historial, agenda, consulta cerrada con diagnóstico, tratamiento y receta (incluido el rollback ante medicamento inexistente), archivo, usuarios y validación de entradas.
- **Aislamiento entre instituciones** (52 checks dentro de la misma corrida): lectura o búsqueda por id de otra institución, headers y query falsos, token manipulado, `alg=none`, endpoints de especialidad desde otra institución, y que un administrador normal no pueda cambiar de institución.
- **Persistencia tras reinicio** (`api-audit.mjs --verify`): después de reiniciar la API y los contenedores, los datos creados siguen en los cuatro motores. Además, `web-smoke.mjs` (Playwright) recorre el login y las pantallas buscando errores de consola.

Los resultados de cada corrida quedan en `tests/e2e-audit/results/`.

Fase	Resultado	Archivo
Flujo completo	566/579 PASS, 0 respuestas 5xx (corrida `2611091735`, 9 nov 2026)	`results/flow-2611091735.json`
Aislamiento entre instituciones	52/52 PASS	mismo archivo (áreas `cross *`, `cross-kind`)
Persistencia tras reinicio	17/17 PASS (corrida de referencia `20261109D`)	`results/verify-20261109D.json`
Web smoke	Sin errores de JavaScript; sin respuestas 403 en la ficha para 10 usuarios demo	`results/web-smoke.json`

Los 13 FAIL restantes de la corrida `2611091735`:

- 4: el rol `administrador` todavía puede registrar consulta con diagnóstico y receta. Es una decisión de roles pendiente: la rúbrica no exige separarlo.
- 4: el access token sigue siendo válido hasta 30 min después del logout. El JWT no guarda estado; el refresh sí se revoca. Está documentado en `docs/ARCHITECTURE.md`.
- 3: la auditoría usaba al médico demo para odontología, que exige el rol `odontologo`. Ya usa `demo-4-odontologo`.
- 2: aplicar una dosis de vacuna exigía `id_profesional` en el cuerpo. Ahora se toma del usuario autenticado. Estos 5 casos quedan pendientes de volver a correr cuando se restablezca la cuota diaria de Firestore.

---

8. Equivalencia de operaciones y límites de evidencia

Las operaciones nucleares del historial (buscar y registrar paciente, archivar, cerrar consulta con diagnóstico, tratamiento y receta) se exponen por la misma API y se ejecutan de forma equivalente en los cuatro adaptadores (contrato C3, suite C3.4, §6). Las especialidades se montan en el motor asignado: hospitalización en PostgreSQL, pediatría en MySQL, laboratorio en MongoDB y odontología en Firestore (ver `docs/ARCHITECTURE.md`, lanes 1–2).

Límites explícitos (no inventados en este documento):

- El cifrado en reposo y el cifrado a nivel de campo quedan fuera del alcance del MVP (§4.1). Solo Firestore en modo real queda cifrado en reposo, y eso lo hace Google por defecto.
- Las reglas de Firestore no aseguran el Admin SDK; la seguridad clínica odontológica depende del backend JWT.
- En MongoDB y Firestore, la búsqueda por nombre se filtra en el adaptador y no en el motor (§6.1).
- Este documento describe decisiones de modelado, consistencia y seguridad presentes en los ADR, los contratos y el código de los adaptadores. No sustituye las capturas de UI ni el checklist de la demostración oral.

Referencias rápidas

Tema	Ruta
Asignación	`docs/specs/adr/ADR-0002-institution-engines.md`, `shared/src/engines.ts`
Aggregates	`ADR-0003`, `ADR-0004`, `docs/plan/contracts/DOCUMENTS-SECURITY.md`
Seguridad	`ADR-0005`, `SECURITY-FREE.md`, `*/security.sql`, `mongodb/security.ts`, `firestore.rules`
Transacciones	`*/unit-of-work.ts`, `*/consulta-repository.ts`, `application/close-consulta.ts`
Consultas equivalentes	`*/paciente-repository.ts`, `*/consulta-repository.ts`, `infra/firebase/firestore.indexes.json`
Resultados lab	`adapters/mongodb/full/lab-results.ts`, `schema.json`
Auditoría	`tests/e2e-audit/README.md`, `tests/e2e-audit/results/`

Documento breve de Entrega II. No afirma despliegue productivo.